UNIVERSIDAD MILITAR NUEVA GRANADA



Facultad de Ingeniería
Programa de Ingeniería Mecatrónica
Asignatura: Realidad Virtual

Informe de Práctica de Laboratorio

Teleoperación de Robot Móvil en Entorno Virtual Inmersivo con Control Háptico Inalámbrico

Integrantes:

David Stiven León Paiva
Cristian David Rosales Hernández
José Luis Barragán Rivera

29 de septiembre de 2026

Contents

1 Introducción **3**

2 Objetivos **3**
2.1 Objetivo general . . . 3
2.2 Objetivos específicos . . . 3

3 Marco teórico **4**
3.1 Realidad virtual y teleoperación . . . 4
3.2 Unidad de medición inercial (IMU) y filtro complementario . . . 4
3.3 Comunicación Bluetooth . . . 4
3.4 Modulación por ancho de pulso (PWM) y actuador vibrotáctil . . . 4
3.5 Microcontrolador ESP32 . . . 5
3.6 Transistor NPN como interruptor electrónico . . . 5
3.7 Diodo de protección . . . 5
3.8 Fuente de alimentación . . . 5
3.9 Visualización inmersiva mediante teléfono celular (VRidge) . . . 5

4 Arquitectura general del sistema **5**

5 Desarrollo del entorno virtual en Unity **7**
5.1 Movimiento del robot móvil . . . 9
5.2 Sistema de agarre de objetos . . . 9

6 Diseño electrónico del control háptico **9**
6.1 Componentes utilizados . . . 9
6.2 Circuito de control del actuador vibrotáctil . . . 10
6.3 Conexión de los pulsadores . . . 10
6.4 Alimentación . . . 11
6.5 Diseño conceptual de la carcasa . . . 11

7 Firmware del ESP32 **13**

8 Integración entre Unity y el control háptico **14**

9 Integración de visualización inmersiva (VRidge) **14**
9.1 Resultados de la integración . . . 15

10 Procedimiento **16**

11 Resultados **16**

12 Análisis de resultados **17**
12.1 Implementación del sistema 17
12.2 Calidad percibida de la retroalimentación háptica 17
12.3 Estabilidad del control 17
12.4 Facilidad de uso del sistema 17

13 Conclusiones **17**

14 Preguntas para la discusión **18**

1 Introducción

La combinación de realidad virtual, teleoperación y sistemas hápticos permite que una persona controle un sistema remoto —físico o virtual— recibiendo, además, información sensorial de vuelta sobre lo que ese sistema está experimentando. En este proyecto se implementó un sistema completo de teleoperación inmersiva en el que un usuario controla un robot móvil dentro de un entorno virtual desarrollado en Unity 3D, utilizando como interfaz de control un dispositivo háptico inalámbrico diseñado y construido por el equipo.

El sistema desarrollado integra cinco componentes que trabajan de forma coordinada: un entorno virtual en primera persona con obstáculos y objetos manipulables; un robot móvil virtual cuyo movimiento se mapea directamente desde la inclinación de la mano del usuario; un control háptico inalámbrico basado en un microcontrolador ESP32 con una unidad de medición inercial (IMU); un enlace de comunicación Bluetooth bidireccional entre el control y el computador; y un actuador vibrotáctil capaz de generar dos patrones de vibración distintos según el tipo de colisión detectada en el entorno virtual. Adicionalmente, se exploró la visualización inmersiva del entorno mediante un teléfono celular como visor tipo Cardboard, usando la aplicación VRidge para transmitir la imagen renderizada por Unity y el giroscopio del teléfono como fuente de seguimiento de cabeza.

Este documento describe el marco teórico de los elementos utilizados, la arquitectura general del sistema, el diseño electrónico y mecatrónico del control háptico, el desarrollo del software tanto en Unity como en el microcontrolador, los resultados obtenidos durante las pruebas de implementación, y las dificultades técnicas encontradas —particularmente en la integración del visor mediante VRidge— junto con las soluciones intentadas.

2 Objetivos

2.1 Objetivo general

Diseñar e implementar un sistema de teleoperación inmersivo en realidad virtual que permita controlar un robot móvil mediante un dispositivo háptico inalámbrico desarrollado por el equipo.

2.2 Objetivos específicos

- Diseñar e implementar en Unity 3D un entorno virtual inmersivo con navegación en primera persona, límites físicos definidos, un robot móvil funcional y un sistema de interacción para la manipulación de objetos.
- Diseñar y construir un dispositivo de control inalámbrico basado en microcontrolador, con IMU, botones de navegación y agarre, actuador vibrotáctil controlado por PWM, y alimentación mediante batería recargable.
- Implementar la comunicación inalámbrica por Bluetooth entre el control háptico y el entorno virtual, incluyendo el mapeo de los ángulos de inclinación hacia las variables de movimiento del robot.

- Validar el sistema mediante pruebas de implementación que permitan analizar el cumplimiento de los requerimientos funcionales planteados.

3 Marco teórico

3.1 Realidad virtual y teleoperación

La realidad virtual (VR) es una tecnología que genera un entorno simulado con el que un usuario puede interactuar de forma inmersiva, típicamente a través de un visor que ocupa su campo visual y dispositivos de entrada que capturan sus movimientos o acciones. La teleoperación, por su parte, consiste en el control de un sistema remoto —en este caso, un robot dentro de un entorno virtual— a partir de las acciones de un operador humano, habitualmente con algún tipo de retroalimentación sensorial que le permita percibir el estado de ese sistema remoto sin observarlo directamente de forma física.

3.2 Unidad de medición inercial (IMU) y filtro complementario

Una IMU es un sensor que combina un acelerómetro y un giroscopio para estimar la orientación de un objeto en el espacio. El acelerómetro es preciso en reposo pero sensible al ruido y a las vibraciones bruscas; el giroscopio, en cambio, responde de forma suave a los movimientos pero acumula un error de deriva (*drift*) con el tiempo. En este proyecto se utilizó un sensor MPU6050 y se combinaron ambas fuentes mediante un **filtro complementario**, que pondera la integración del giroscopio a corto plazo con la corrección lenta del acelerómetro.

3.3 Comunicación Bluetooth

Se empleó el perfil **Bluetooth Clásico SPP** (*Serial Port Profile*), disponible de forma nativa en el ESP32 a través de la librería `BluetoothSerial`. Este perfil emula un puerto serial tradicional sobre un enlace inalámbrico: al emparejar el dispositivo con el computador, el sistema operativo crea automáticamente un puerto COM virtual que puede leerse y escribirse exactamente igual que un cable serial físico, lo que simplificó considerablemente la integración con Unity.

3.4 Modulación por ancho de pulso (PWM) y actuador vibrotáctil

La retroalimentación háptica se implementó mediante un motor vibrador de corriente continua (tipo *coin motor*, de dos cables), controlado por una señal PWM generada desde el ESP32. Variando el ciclo de trabajo y la duración de los pulsos fue posible generar dos patrones de vibración perceptualmente distintos: uno corto y de intensidad media (asociado a la colisión con un objeto manipulable) y otro de mayor intensidad, con varios pulsos intermitentes (asociado a la colisión con una pared o límite del entorno).

3.5 Microcontrolador ESP32

El ESP32 es un microcontrolador de bajo costo con conectividad WiFi y Bluetooth integrada, múltiples periféricos (I2C, PWM, entradas digitales con resistencia de *pull-up* interna) y suficiente capacidad de procesamiento para ejecutar simultáneamente la lectura de sensores, el filtrado de señales y la comunicación inalámbrica. Fue el elemento central del control háptico desarrollado.

3.6 Transistor NPN como interruptor electrónico

Dado que las salidas digitales del ESP32 no pueden entregar la corriente que requiere un motor vibrador de forma segura, se utilizó un **transistor NPN 2N2222** operando en la región de corte/saturación, es decir, como un interruptor controlado electrónicamente. Una señal PWM de baja corriente aplicada a la base (a través de una resistencia de 1 kΩ) permite o interrumpe el paso de una corriente mucho mayor entre el colector y el emisor, alimentando así el motor sin exponer al microcontrolador a una carga eléctrica excesiva.

3.7 Diodo de protección

En paralelo con el motor se instaló un **diodo** (tipo Zener/rectificador de propósito general) cuya función es absorber el pico de tensión inversa que un motor de corriente continua genera al interrumpirse abruptamente su alimentación. Sin esta protección, dicho pico podría dañar progresivamente el transistor de conmutación.

3.8 Fuente de alimentación

El control háptico se alimentó con una **batería recargable de litio (LiPo, 3.7 V)**, gestionada por un **módulo de carga** que permite recargarla mediante un puerto USB sin necesidad de retirarla del circuito. La tensión de la batería se conectó directamente al pin de 3.3 V del ESP32 (evitando el regulador interno del pin VIN, que requiere una tensión de entrada mayor para funcionar de forma estable).

3.9 Visualización inmersiva mediante teléfono celular (VRidge)

Como alternativa al uso de un visor VR dedicado, se exploró el uso de un teléfono celular Android como visor tipo Cardboard, empleando el software **VRidge** (RiftCat). Esta herramienta permite que un teléfono actúe como fuente de seguimiento de cabeza (mediante su giroscopio) y como pantalla estereoscópica, transmitiendo por red local la imagen renderizada por el computador a través de una integración con SteamVR y el estándar OpenXR/OpenVR.

4 Arquitectura general del sistema

La Figura 2 resume el flujo de información entre los componentes del sistema: el control háptico mide la inclinación de la mano y el estado de sus botones, y transmite esta información por Bluetooth al computador; Unity la utiliza para mover el robot móvil virtual y desplazar

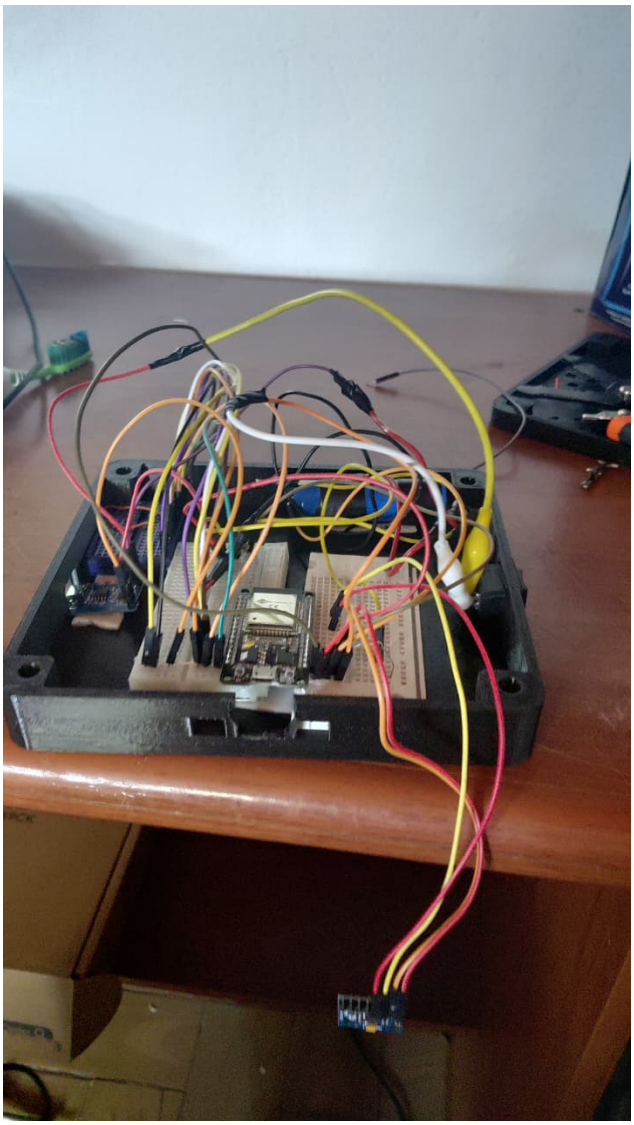


Figure 1: Montaje electrónico del control háptico sobre protoboard (pendiente de fotografía real

al avatar del usuario; cuando el robot colisiona con un objeto o con un límite del entorno, Unity envía de vuelta un comando por el mismo canal Bluetooth para activar el patrón de vibración correspondiente.

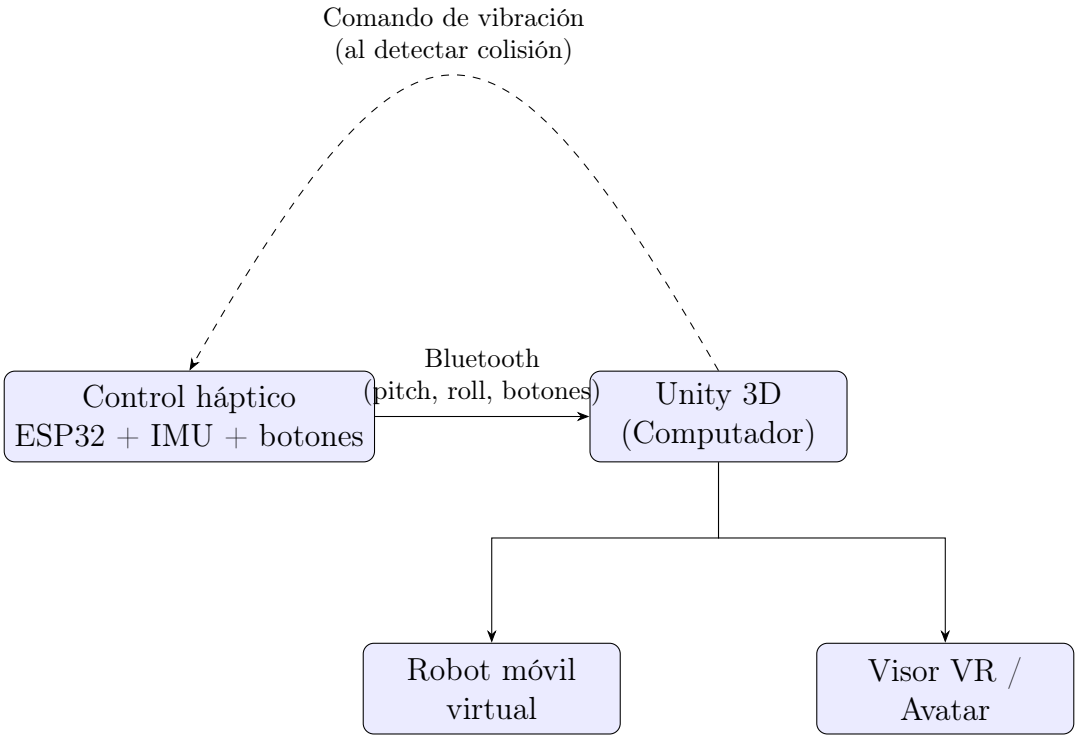


Figure 2: Arquitectura general del sistema de teleoperación.

5 Desarrollo del entorno virtual en Unity

Se construyó un entorno virtual en Unity 3D compuesto por un piso de 30×30 unidades, cuatro paredes que delimitan el área, tres obstáculos tipo columna, y tres objetos manipulables (dos esferas y un cubo). Sobre este escenario se implementó un avatar en primera persona (un `Character Controller` con una cámara asociada), un robot móvil independiente con física propia, y un sistema de agarre de objetos basado en detección de proximidad.

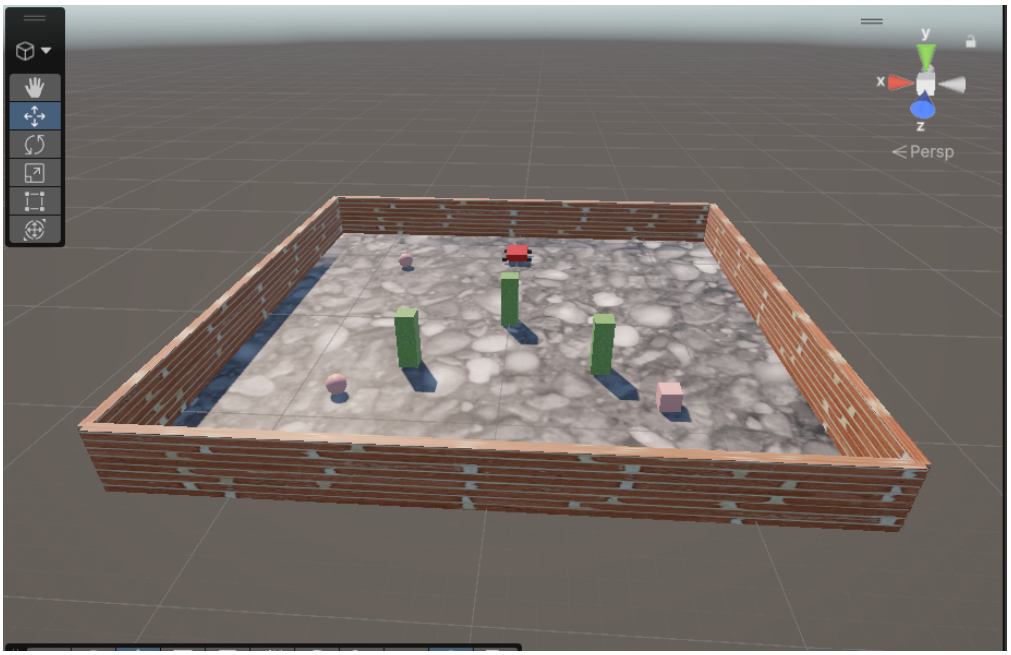


Figure 3: Vista general del entorno virtual desarrollado, mostrando las paredes, obstáculos, objetos manipulables y el robot móvil.

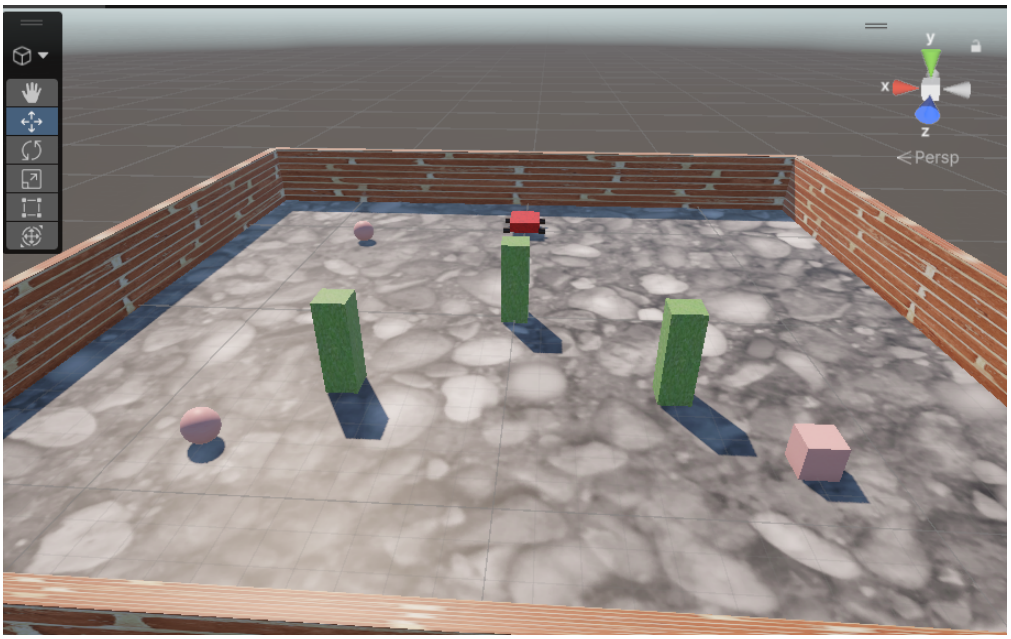


Figure 4: Vista alternativa del entorno virtual.

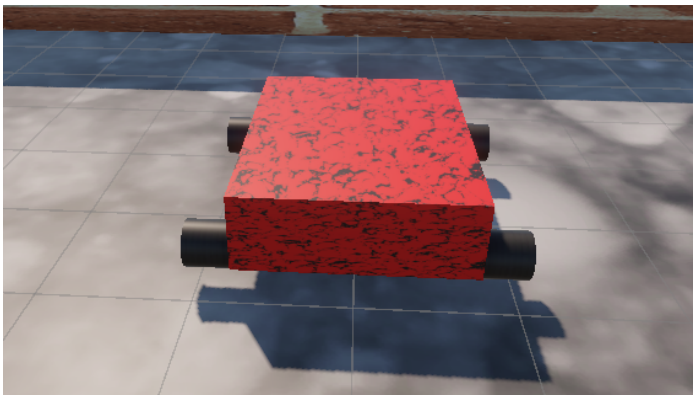


Figure 5: Modelo del robot móvil virtual, construido a partir de primitivas geométricas (cuerpo y cuatro ruedas).

5.1 Movimiento del robot móvil

El movimiento del robot se implementó mediante un `Rigidbody` controlado por código, siguiendo el mapeo definido: la inclinación de *Pitch* controla el avance/retroceso, y la de *Roll* controla el giro. El siguiente fragmento resume la lógica de movimiento:

```
Vector3 avance = transform.forward * pitchSimulado
                 * velocidadMax * Time.fixedDeltaTime;
rb.MovePosition(rb.position + avance);

float rotacionGrados = rollSimulado * giroMax * Time.fixedDeltaTime;
rb.MoveRotation(rb.rotation * Quaternion.Euler(0f, rotacionGrados, 0f));
```

Listing 1: Mapeo de Pitch/Roll al movimiento del robot (C#, Unity).

Adicionalmente, el robot detecta colisiones y notifica al control háptico por Bluetooth el tipo de superficie con la que impactó (objeto manipulable o límite/pared), utilizando etiquetas (*tags*) asignadas a cada tipo de objeto en la escena.

5.2 Sistema de agarre de objetos

Se implementó un punto de agarre ubicado frente a la cámara del avatar, con un *collider* en modo *trigger* que detecta la proximidad de objetos etiquetados como "Manipulable". Al presionar el botón de agarre, el objeto detectado se vuelve hijo del punto de agarre y su componente `Rigidbody` se marca como cinemático (dejando de responder a la gravedad); al soltar, recupera su física normal.

6 Diseño electrónico del control háptico

6.1 Componentes utilizados

- Microcontrolador ESP32 (con Bluetooth Clásico y WiFi integrados).
- Unidad de medición inercial MPU6050 (acelerómetro + giroscopio, comunicación I2C).

- Cinco pulsadores (W, A, S, D para navegación del avatar, y E para agarre de objetos).
- Transistor NPN 2N2222, utilizado como interruptor electrónico para el motor.
- Diodo de protección (tipo Zener), en paralelo con el motor.
- Resistencia de 1 kΩ, para limitar la corriente de base del transistor.
- Motor vibrador de corriente continua (*coin motor*, dos cables).
- Batería recargable de litio 3.7 V y módulo de carga (tipo TP4056).
- Interruptor de encendido/apagado general.
- Protoboard, para el montaje y prueba de las conexiones.
- Cable UTP, utilizado como conductor para las interconexiones del circuito.

6.2 Circuito de control del actuador vibrotáctil

El motor no puede conectarse directamente a un GPIO del ESP32, ya que su consumo de corriente excede lo que un pin puede entregar de forma segura. Se diseñó el circuito de la Figura 6, en el que el GPIO controla, a través de una resistencia de 1 kΩ, la base de un transistor NPN 2N2222 que actúa como interruptor entre la batería y el motor.

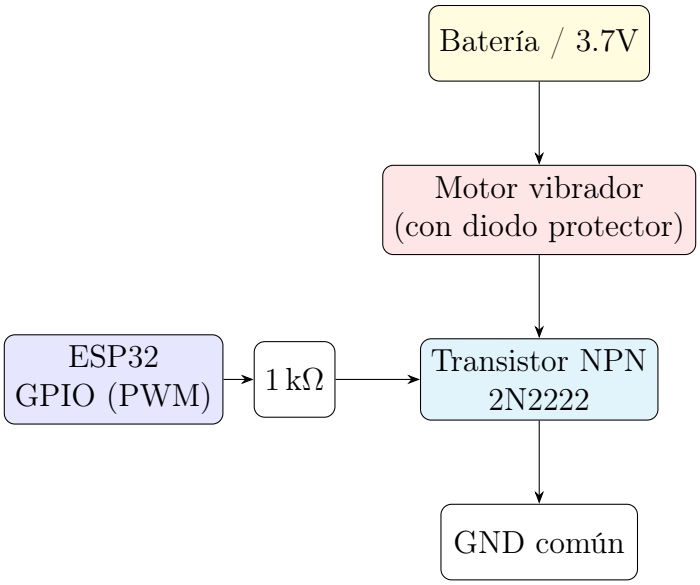


Figure 6: Circuito de control del motor vibrador mediante transistor NPN.

6.3 Conexión de los pulsadores

Los cinco pulsadores se conectaron entre un pin GPIO y GND, utilizando la resistencia de *pull-up* interna del ESP32 (modo `INPUT_PULLUP`), de forma que el estado "presionado" corresponde a una lectura en bajo (`LOW`). La Tabla 1 resume la asignación de pines utilizada.

Table 1: Asignación de pines GPIO del ESP32.

Función	Pin GPIO
IMU (SDA)	21
IMU (SCL)	22
Motor vibrador (PWM)	4
Botón W (avanzar avatar)	13
Botón A (izquierda)	14
Botón S (retroceder avatar)	27
Botón D (derecha)	26
Botón E (agarrar)	25

6.4 Alimentación

La batería de 3.7 V se conectó a las terminales de un módulo de carga, que permite recargarla mediante USB sin desconectarla del circuito. La alimentación del ESP32 se tomó directamente al pin de 3.3 V (sin pasar por el regulador interno del pin VIN, que requiere una tensión de entrada superior a la de la batería para operar establemente), compartiendo el mismo riel positivo con el circuito del motor.

6.5 Diseño conceptual de la carcasa

Se elaboró un diseño conceptual de la carcasa que alberga el circuito, pensado para imprimirse en 3D, con cuatro botones de navegación dispuestos en cruz, un botón adicional de agarre, y un interruptor general de encendido accesible en uno de los costados. Las Figuras 7 y 8 muestran un **render conceptual de referencia** de este diseño (pendiente de reemplazar por fotografías de la pieza impresa y ensamblada).



Figure 7: Render conceptual de la carcasa del control háptico, vista superior (diseño de referencia).



Figure 8: Render conceptual de la carcasa del control háptico, vista superior (diseño de referencia).

7 Firmware del ESP32

El firmware del control háptico se estructuró en las siguientes etapas, ejecutadas de forma cíclica:

1. **Calibración inicial:** al encender el dispositivo, se promedian 200 muestras del acelerómetro y el giroscopio con el sensor en reposo, para determinar y compensar el sesgo (*offset*) propio de cada sensor.

2. **Lectura y filtrado:** en cada ciclo se leen el acelerómetro y el giroscopio, se calculan los ángulos de Pitch y Roll a partir del acelerómetro, y se combinan con la integración del giroscopio mediante el filtro complementario descrito en el marco teórico.

3. **Lectura de botones:** se revisan los cinco pulsadores (W, A, S, D, E) mediante lectura digital directa.

4. **Transmisión por Bluetooth:** se envía una línea de texto con el formato `pitch,roll,w,a,s,d,e`, donde los últimos cinco valores son binarios (1 = presionado, 0 = no presionado).

5. **Recepción de comandos:** en paralelo, el firmware revisa si llegó algún comando desde Unity (`'1'` o `'2'`) para activar el patrón de vibración correspondiente.

```
pitchFiltrado = ALPHA * (pitchFiltrado + gyroYgrados * dt)
                + (1 - ALPHA) * pitchAcc;
rollFiltrado  = ALPHA * (rollFiltrado  + gyroXgrados * dt)
                + (1 - ALPHA) * rollAcc;

SerialBT.print(pitchFiltrado);   SerialBT.print(",");
SerialBT.print(rollFiltrado);    SerialBT.print(",");
SerialBT.print(w); SerialBT.print(","); SerialBT.print(a); SerialBT.print(
    ",");
SerialBT.print(s); SerialBT.print(","); SerialBT.print(d); SerialBT.print(
    ",");
SerialBT.println(e);
```

Listing 2: Fragmento del firmware: filtro complementario y transmisión por Bluetooth.

8 Integración entre Unity y el control háptico

Del lado de Unity, se desarrolló un script (`ConexionBluetooth`) que abre el puerto COM virtual creado por el emparejamiento Bluetooth y lee continuamente, en un hilo independiente, las líneas enviadas por el ESP32. Los valores recibidos se exponen a los demás scripts del proyecto (control del robot, movimiento del avatar y sistema de agarre), reemplazando progresivamente las entradas de teclado utilizadas durante las primeras pruebas.

```
string linea = puerto.ReadLine();
string[] partes = linea.Split(',');
if (partes.Length == 7) {
    pitch = float.Parse(partes[0], CultureInfo.InvariantCulture);
    roll  = float.Parse(partes[1], CultureInfo.InvariantCulture);
    botonW = partes[2].Trim() == "1";
    // ... análogo para A, S, D, E
}
```

Listing 3: Lectura del puerto Bluetooth en un hilo aparte (C#, Unity).

De forma complementaria, cuando el robot móvil colisiona con un objeto etiquetado como “Manipulable” o como “Limite”, Unity envía por el mismo puerto el comando `'1'` o `'2'` respectivamente, cerrando el lazo de retroalimentación háptica.

9 Integración de visualización inmersiva (VRidge)

Como complemento al entorno desarrollado, se intentó integrar un teléfono celular Android como visor VR tipo Cardboard, utilizando el software VRidge para transmitir la imagen

renderizada por Unity y recibir de vuelta la orientación del teléfono (giroscopio) como fuente de seguimiento de cabeza. Para ello se instaló SteamVR y VRidge en el computador, la aplicación complementaria de VRidge en el teléfono, y se habilitó en Unity el paquete OpenXR dentro de *XR Plug-in Management*, junto con un componente `Tracked Pose Driver` enlazado a la posición y rotación del "ojo central" (*centerEye*) del visor detectado.

9.1 Resultados de la integración

Se logró establecer la conexión entre el teléfono y el computador, confirmándose mediante la vista previa de VRidge que el giroscopio del teléfono transmitía correctamente su orientación, la cual se reflejaba de forma fluida en la rotación de la cámara dentro del editor de Unity. Sin embargo, al observar la imagen a través del propio teléfono (dentro del visor), **la escena permanecía fija**: aunque el movimiento de la cabeza se recibía correctamente en el computador, la imagen reenviada al teléfono no se actualizaba en consecuencia.

Se descartaron progresivamente varias causas: el estado de pausa del editor, la pérdida de foco de la ventana de Unity (para lo cual se activó la opción *Run In Background*), y la ejecución mediante un ejecutable compilado en lugar del modo de reproducción del editor. Ninguna de estas medidas resolvió el problema. La hipótesis más probable es una **incompatibilidad entre el puente OpenXR-sobre-SteamVR y VRidge**, dado que este último fue diseñado originalmente sobre la interfaz más antigua de SteamVR (OpenVR) y no necesariamente sobre el estándar OpenXR más reciente. Se intentó instalar el paquete de compatibilidad OpenVR directamente en Unity como medida adicional; este resultado quedó pendiente de confirmación al cierre de este informe.

Esta dificultad no impidió cumplir el objetivo de navegación en primera persona, ya que el sistema cuenta con un modo de respaldo completamente funcional mediante mouse y teclado (o los botones físicos del control háptico), que fue el utilizado para las pruebas de validación del resto del sistema.

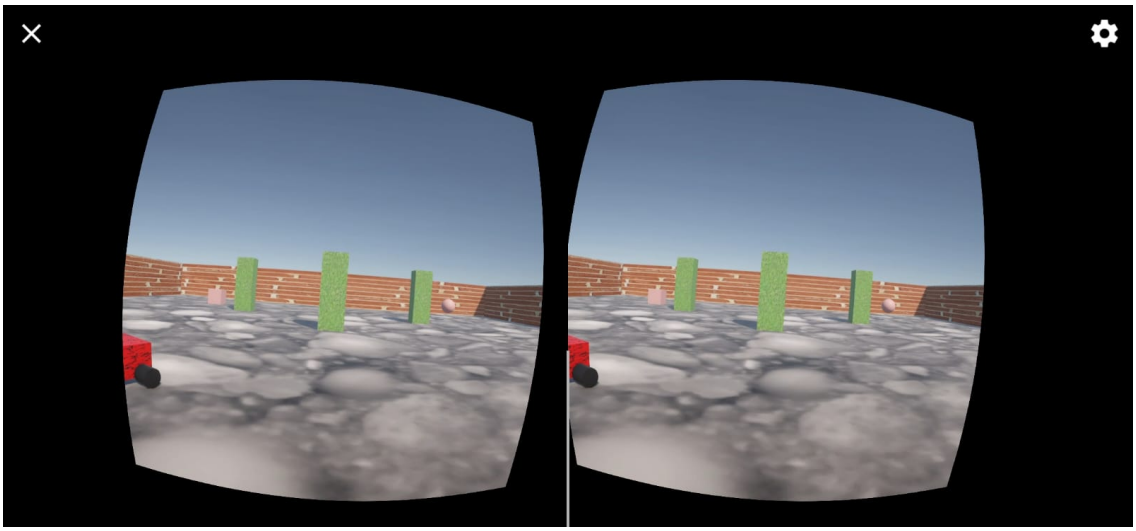


Figure 9: Vista estereoscópica recibida en el teléfono a través de VRidge, mostrando el entorno virtual correctamente renderizado (problema de actualización en tiempo real aún en investigación).

10 Procedimiento

El desarrollo del sistema se organizó en cinco fases:

1. **Diseño conceptual:** definición de la arquitectura general del sistema y del mapeo de variables (Pitch/Roll hacia el movimiento del robot; botones hacia la navegación del avatar y el agarre de objetos).
2. **Desarrollo del entorno virtual:** construcción del escenario en Unity, configuración de la navegación en primera persona, implementación de los límites físicos, del robot móvil con física propia, y del sistema de agarre de objetos.
3. **Desarrollo del control háptico:** selección y calibración de la IMU, implementación del filtro complementario, programación de la transmisión Bluetooth, y del control PWM con patrones diferenciados de vibración.
4. **Manufactura y ensamble:** diseño conceptual de la carcasa para impresión 3D e integración de la electrónica y la batería.
5. **Integración y pruebas:** conexión de Unity con el control háptico por Bluetooth, validación del mapeo de movimiento, ajuste de sensibilidad, y validación de la retroalimentación háptica ante colisiones; adicionalmente, se exploró la integración de un visor VR mediante teléfono celular y VRidge.

11 Resultados

Al finalizar el desarrollo, el sistema presenta las siguientes funcionalidades verificadas:

- Entorno virtual funcional, con navegación en primera persona, límites físicos y obstáculos correctamente delimitados.
- Robot móvil virtual que responde de forma proporcional a la inclinación (Pitch/Roll) del control háptico, transmitida en tiempo real por Bluetooth.
- Sistema de agarre de objetos operado mediante un botón físico del control háptico.
- Navegación del avatar controlada mediante cuatro botones físicos (W, A, S, D).
- Retroalimentación háptica diferenciada: un patrón corto y suave ante la colisión con un objeto manipulable, y un patrón fuerte e intermitente ante la colisión con una pared o límite del entorno.
- Comunicación Bluetooth bidireccional estable entre el ESP32 y Unity, sin pérdida perceptible de datos durante las pruebas realizadas.
- Visualización inmersiva mediante teléfono celular parcialmente lograda: el seguimiento de cabeza (giroscopio) funciona correctamente, pero la actualización de la imagen en el propio visor presentó una limitación técnica no resuelta al momento de este informe.

12 Análisis de resultados

12.1 Implementación del sistema

El sistema logró integrar de forma coordinada un entorno virtual, un dispositivo mecatrónico y un enlace de comunicación inalámbrica, cumpliendo con la mayoría de los requerimientos planteados en la guía de práctica. La principal limitación encontrada fue la integración del visor mediante teléfono celular, cuya causa raíz se atribuye a una incompatibilidad entre el software VRidge y la capa de compatibilidad OpenXR usada por Unity para comunicarse con SteamVR.

12.2 Calidad percibida de la retroalimentación háptica

Los dos patrones de vibración implementados (corto/suave para objetos manipulables, e intermitente/fuerte para paredes) resultaron distinguibles al tacto tras algunos ajustes en la intensidad y duración de los pulsos, cumpliendo su propósito de comunicar al usuario, sin necesidad de mirar el motor, qué tipo de colisión ocurrió.

12.3 Estabilidad del control

El filtro complementario aplicado a las lecturas de la IMU eliminó de forma efectiva el temblor característico de una lectura basada únicamente en el acelerómetro, permitiendo un control de movimiento suave y predecible del robot móvil. La comunicación Bluetooth se mantuvo estable durante las sesiones de prueba, sin desconexiones inesperadas.

12.4 Facilidad de uso del sistema

El mapeo de Pitch/Roll hacia el movimiento del robot resultó intuitivo tras un breve periodo de ajuste de la sensibilidad (mediante el parámetro de ángulo máximo). La separación entre el control del robot (por inclinación) y la navegación del avatar (por botones físicos) permitió operar ambos sin confusión, replicando el diseño conceptual planteado en la guía de práctica, en la que el usuario observa y opera al robot como un elemento independiente dentro del entorno.

13 Conclusiones

El desarrollo de este sistema de teleoperación inmersivo permitió integrar, en un mismo proyecto, conocimientos de programación de entornos virtuales, diseño mecatrónico, comunicaciones inalámbricas y sistemas hápticos. El uso de un filtro complementario resultó esencial para obtener una señal de control estable a partir de una IMU de bajo costo, y el diseño incremental —probando primero cada subsistema con entradas simuladas por teclado antes de conectar el hardware real— facilitó considerablemente la identificación de errores durante la integración.

La principal lección técnica del proyecto estuvo relacionada con la integración de la visualización inmersiva: aunque el seguimiento de cabeza mediante teléfono celular se logró transmitir correctamente, la actualización de la imagen en el visor no se resolvió por completo, evidenciando que la compatibilidad entre herramientas de terceros (como VRidge) y los estándares más recientes de la industria (OpenXR) no siempre está garantizada, y que la exploración de tecnologías con menor documentación disponible puede introducir riesgos considerables en el cronograma de un proyecto de este tipo.

En conjunto, el sistema desarrollado cumple con los objetivos centrales planteados —control remoto de un robot móvil mediante un dispositivo háptico propio, con retroalimentación diferenciada— y sienta una base sólida para futuros desarrollos en teleoperación, robótica y simulación inmersiva.

14 Preguntas para la discusión

¿Cómo afecta la latencia del Bluetooth a la percepción de control en tiempo real?
Durante las pruebas no se percibió una latencia notable en el movimiento del robot; el perfil SPP de Bluetooth Clásico, al comportarse como un puerto serial convencional, introduce una latencia baja y consistente, adecuada para esta aplicación.

¿Qué tipo de filtro es más adecuado para estabilizar la IMU en este tipo de aplicación?
El filtro complementario utilizado resultó suficiente y computacionalmente económico para el ESP32. Un filtro de Kalman podría ofrecer una estimación aún más precisa, a costa de mayor complejidad de implementación y carga de procesamiento.

¿Qué ventajas tendría implementar control proporcional en lugar de control por umbrales?
El sistema ya implementa, de hecho, un control proporcional: la velocidad y el giro del robot son proporcionales al ángulo de inclinación medido, dentro de un rango máximo configurable, lo cual permite un control más fino que un simple esquema de encendido/apagado por umbral.

¿Cómo cambiaría el diseño si el robot fuera físico y no virtual?
Sería necesario reemplazar el `Rigidbody` virtual por un sistema de tracción real (motores DC o motores paso a paso), y considerar la latencia de un segundo enlace inalámbrico entre el computador y el robot físico, además de mecanismos de seguridad ante pérdida de señal.

¿Qué variables influyen en la percepción diferenciada de vibraciones?
La duración del pulso, su intensidad (ciclo de trabajo del PWM), y el patrón temporal (pulso único frente a pulsos intermitentes) fueron las variables ajustadas para lograr que los dos patrones implementados resultaran distinguibles al tacto.

¿Cómo se podría implementar retroalimentación háptica más rica (fuerza, no solo vibración)?
Se requeriría un actuador de retroalimentación de fuerza (por ejemplo, un motor con reductor y control de par, o un sistema de frenado magnético), capaz de oponer resistencia física al movimiento del usuario, en lugar de únicamente generar vibración.

¿Qué implicaciones tendría este sistema en aplicaciones médicas o industriales?
El mismo principio de teleoperación con retroalimentación háptica es aplicable a la ma-

nipulación remota de instrumental quirúrgico o a la operación de maquinaria en entornos peligrosos, donde la percepción táctil de colisiones o resistencias puede prevenir daños al equipo o lesiones al operador.

¿Cómo se podría mejorar la inmersión del sistema?
Resolviendo la limitación identificada en la integración con VRidge (o utilizando en su lugar un visor con soporte nativo de OpenXR, como el Meta Quest 3 disponible en el laboratorio), y agregando seguimiento de manos o controles adicionales para enriquecer la interacción con los objetos manipulables.

References

[1] LaValle, S. M. (2023). *Virtual Reality*. Cambridge University Press.

[2] Sherman, W. R., & Craig, A. B. (2018). *Understanding Virtual Reality: Interface, Application, and Design*. Morgan Kaufmann.